

四、城市交通

在当今快节奏的都市生活中，**通勤**已经成为城市居民日常生活中不可或缺的一部分。尤其是在中国的一线城市，如北京、上海、广州和深圳，这些城市的经济高度发达，人口密集，通勤问题尤为突出。通勤不仅影响着居民的生活质量，还对城市的交通管理、环境保护以及能源消耗等方面产生深远的影响。因此，深入分析一线城市上班群体的通勤情况，对于优化城市交通规划、提升居民生活品质以及推动可持续发展具有重要的现实意义。

在一线城市，最常见的两种出行方式是**地铁**和**汽车**。这两种出行方式各有特点，也分别反映了城市交通的两个重要方面：公共交通的便利性和私人交通的效率。

地铁作为一种高效的公共交通工具，以其快速、准时、大运量的特点，在城市交通中扮演着重要角色。地铁站的分布情况直接关系到居民选择地铁出行的便利程度。密集的地铁站分布可以为居民提供更多的出行选择，减少步行距离，提高出行效率。因此，通过分析**地铁的分布情况**，我们可以直观地了解城市低碳出行的便捷程度。

另一方面，汽车作为私人交通工具，虽然在灵活性和舒适性上具有优势，但在上下班高峰期，道路交通的拥挤程度往往会严重影响汽车的通行效率。尤其是在一线城市，由于人口密集、车辆保有量高，交通拥堵问题已经成为制约城市发展的一大瓶颈。通过分析各个城市**道路交通的拥挤度**，我们可以直观地了解汽车在上下班高峰时段的通行便捷程度，以及城市交通管理的现状和挑战。

下文将从地铁站分布情况，与全天城市道路拥挤程度分析两方面，衡量北上广深四个城市的通勤便捷程度，从而比较四个城市的生活质量。

4.1 地铁站分布情况分析

4.1.1 数据获取

（1）爬取目标与背景

地铁作为城市交通的重要组成部分，其站点分布和线路信息对于居民出行的便利程度具有重要意义。为了获取北上广深四个城市的地铁站点信息，本文选择**高德地图地铁图页面**（<http://map.amap.com/subway/index.html>）作为数据源。该页面提供了丰富的城市地铁线路和站点信息，能够满足研究需求。

（2）爬取方式

➤ 数据结构分析

在开始爬取之前，需要先分析高德地图地铁图页面的数据结构，以便确定如何提取所需信息。

● 页面结构分析：

高德地图地铁图页面的**城市信息**存储在两个 `div` 标签中，分别是 `class="city-list fl"` 和 `class="more-city-list"`。这两个标签分别包含了主要城市和更多城市的列表。每个城市的信息包括城市名称、拼音和 ID。例如北京的 HTML 结构如下，`<a id="110000" cityname="beijing">北京</a>` 其中，`id` 属性是城市 ID，`cityname` 属性是城市拼音，文本内容是城市名称。

- 地铁数据请求分析：

通过分析，发现每个城市的地铁数据可以通过构造特定的 URL（如 `http://map.amap.com/service/subway?_1818387860087&srhdata=<city_id>_drw_<city_name>.json`）以 JSON 格式获取。在每个 URL 中，<city_id>和<city_name>分别为城市对应的 ID 和拼音名称。这种结构使得我们可以动态地构造 URL，从而获取不同城市的地铁数据

➤ 爬取流程

第一步：城市信息获取

首先，需要获取所有目标城市的名称、拼音和 ID 信息。这些信息将用于后续构造地铁数据的请求 URL。

- 发送请求：

使用 `requests` 库发送 GET 请求获取高德地图地铁图页面的 HTML 内容。这是爬取的第一步，确保我们能够成功访问目标页面。

```
url = 'http://map.amap.com/subway/index.html'
response = requests.get(url)
```

- 解析网页：

如果请求成功（即状态码为 200），使用 `BeautifulSoup` 解析网页内容。

```
if response.status_code == 200:
    response_content = response.content.decode('utf-8')
    soup = BeautifulSoup(response_content, 'html.parser')
```

- 提取城市信息：

通过 BeautifulSoup 找到包含城市信息的 div 标签，并提取每个城市的信息。将城市名称、拼音和 ID 存储到一个列表中，方便后续使用。

```
city_info_list = []
for soup_a in soup.find('div', class_='city-list fl').find_all('a'):
    city_name_py = soup_a['cityname'] # 城市拼音
    city_id = soup_a['id'] # 城市ID
    city_name_ch = soup_a.get_text() # 城市名称（中文）
    city_info_list.append({'name_py': city_name_py, 'id': city_id, 'name_ch':
                           city_name_ch})
```

- 考虑因素：

在这一过程中，需要确保网页结构没有发生变化。如果高德地图更新了页面布局，可能需要重新分析 HTML 结构并调整代码。此外，还需要处理可能出现的网络请求失败情况，例如通过设置超时时间或重试机制。

第二步：城市信息获取

在获取城市信息后，接下来需要爬取每个城市的地铁数据。

- 构造请求 URL：

根据每个城市的 ID 和拼音，构造地铁数据的请求 URL。这是爬取地铁数据的关键步骤，URL 的正确性直接影响到数据获取的成功与否。

```
url = f"http://map.amap.com/service/subway?_1818387860087&srhdata={city_id}_drw_{city_name}.json"
```

- 发送请求并获取数据：

使用 `requests` 库发送 GET 请求获取地铁数据。将返回的 JSON 数据解析为 Python 对象，以便进一步处理。

```
res = requests.get(url)
content = res.content.decode('utf-8')
content_json = json.loads(content)
```

- 提取地铁线路和站点信息：

从解析后的 JSON 数据中提取地铁线路和站点信息。这些信息通常包含线路名称、站点名称、位置坐标等关键字段。

```
line_info_list = content_json.get("l") # 获取所有地铁线路
for line_info in line_info_list:
    line_name = line_info["kn"] # 线路名称
    df_per_zd = pd.DataFrame(line_info["st"]) # 站点信息
    df_per_zd = df_per_zd[['n', 'sl', 'poiid', 'sp']] # 提取关键字段
    df_per_zd['gd经度'] = df_per_zd['sl'].apply(lambda x: x.split(',')[0]) # 提取经度
    df_per_zd['gd纬度'] = df_per_zd['sl'].apply(lambda x: x.split(',')[1]) # 提取纬度
    df_per_zd.drop('sl', axis=1, inplace=True) # 删除原始坐标字段
    df_per_zd['路线名称'] = line_info['ln'] # 添加线路名称
    df_per_zd['城市名称'] = city_name_ch # 添加城市名称
    df_per_zd.rename(columns={"n": "站点名称", "poiid": "POI编号", "sp": "拼音名称"}, inplace=True) # 重命名字段
```

- 考虑因素：

在这一步中，需要确保请求的 URL 格式正确，并且目标服务器返回的数据是有效的。如果返回的数据为空或格式不正确，可能需要进一步检查城市 ID 和拼音是否正确，或者目标服务器是否进行了更新。此外，还需要考虑数据的完整性，确保每个站点的关键信息都被正确提取。

第三步：数据处理与保存

在获取地铁数据后，需要对数据进行处理，并将其保存为便于后续分析的格式。

- 数据聚合：

将每个城市的地铁线路数据合并到一个 DataFrame 中。这是为了方便后续的数据分析和可视化操作。

```
city_lines_list = [] # 用于存储每个线路的数据
for line_info in line_info_list:
    df_per_zd = pd.DataFrame(line_info["st"])
    df_per_zd = df_per_zd[['n', 'sl', 'poiid', 'sp']]
    df_per_zd['gd经度'] = df_per_zd['sl'].apply(lambda x: x.split(',')[0])
    df_per_zd['gd纬度'] = df_per_zd['sl'].apply(lambda x: x.split(',')[1])
    df_per_zd.drop('sl', axis=1, inplace=True)
    df_per_zd['路线名称'] = line_info['ln']
    df_per_zd['城市名称'] = city_name_ch
    df_per_zd.rename(columns={"n": "站点名称", "poiid": "POI编号", "sp": "拼音名称"}, inplace=True)
    city_lines_list.append(df_per_zd)
city_subway_data = pd.concat(city_lines_list, ignore_index=True) # 合并所有线路数据
```

- 保存数据：

将处理后的地铁站点数据保存为 Excel 文件，便于后续的数据分析和可视化。

```
city_subway_data.to_excel(city_name_ch + '.xlsx', sheet_name='Sheet1')
```

- 考虑因素：

在这一步中，需要确保数据的格式化和聚合操作正确无误。例如，**字段的重命名和拆分操作**需要确保数据的完整性和一致性。

- **数据处理重点**

在代码中，对每个城市的地铁站点数据进行了以下处理：

- 提取站点信息：

从 JSON 数据中提取站点名称、位置坐标、所属线路等关键信息。

- 坐标拆分：

将站点的位置坐标（sl 字段）拆分为经度和纬度两个字段。

- 数据格式化：

将站点名称、线路名称等字段进行重命名，统一格式，方便后续分析。

- 数据聚合：

将每个城市的地铁线路数据合并到一个 DataFrame 中，并保存为 Excel 文件。

4.1.2 数据可视化

（1）地铁站点数量和线路数量的统计

- **数据读取与预处理**

为了统计各个城市的地铁站点数量，我们从汇总的 Excel 文件中读取数据。数据包括城市名称和对应的地铁站点名称。通过遍历数据，计算每个城市的站点总数，并将结果存储在字典 city_station_count 中。为了确保统计的准确性，**使用集合 counted_stations 记录已经统计过的站点，避免重复统计**。计算每个城市的地铁线路总数方法同理，将统计得出的站点总数和地铁线路总数共同存入新的 Excel 文件中。

```
# 创建一个集合用于存储已经统计过的站点
counted_stations = set()

# 创建一个字典用于存储每个城市的站点数量
city_station_count = {}

# 遍历数据，计算每个城市的站点总数
for index, row in data.iterrows():
    city = row['城市']
    station = row['地铁站名']

    # 检查是否已经统计过该站点
    if station not in counted_stations:
        # 将站点加入已统计集合中
        counted_stations.add(station)

    # 检查该城市是否已经有站点数量记录
    if city in city_station_count:
        city_station_count[city] += 1
    else:
        city_station_count[city] = 1
```

➤ 地铁站点总数的可视化

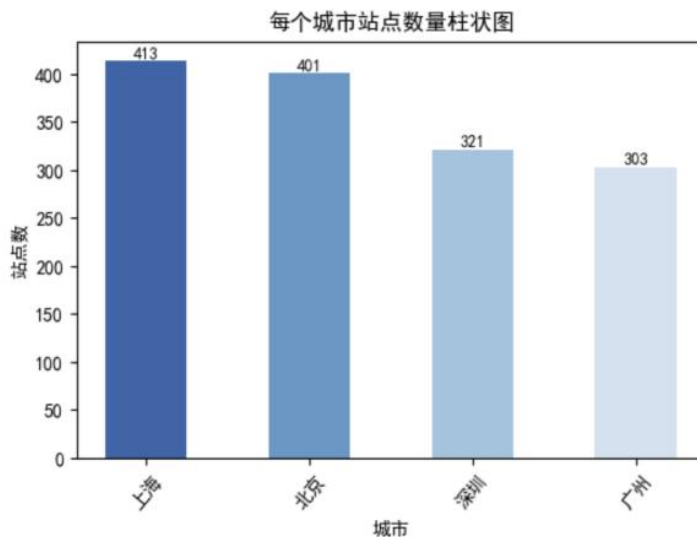
通过 Matplotlib 和 Seaborn 库，完成地铁站点总数的可视化后，我们可以直观地展示各个城市的地铁站点数量。

```
import matplotlib.pyplot as plt
import seaborn as sns

# 将统计结果转换为DataFrame
city_station_df = pd.DataFrame(list(city_station_count.items()), columns=['城市',
'站点数'])

# 使用Seaborn绘制柱状图
plt.figure(figsize=(10, 6))
sns.barplot(x='城市', y='站点数', data=city_station_df, palette='viridis')
plt.title('各城市地铁站点数量对比')
plt.xlabel('城市')
plt.ylabel('站点数')
plt.show()
```

根据统计结果，上海的地铁站点数最多，达到 413 个；而广州的地铁站点数最少，仅有 303 个。这一结果反映了不同城市在地铁建设规模上的差异。上海作为国际化大都市，其地铁网络覆盖范围广、站点密集，能够满足大量居民的出行需求。相比之下，广州的地铁站点数量较少，这可能与其城市的地理面积、人口密度以及经济发展水平有关。广州的城市规划可能更注重其他交通方式的协同发展，或者其地铁建设仍在逐步推进中。



➤ 地铁线路总数的可视化

为了更直观地展示各城市的地铁线路数量分布，我们首先计算地铁线路数量的最大值和最小值，以便设置地图的颜色范围。随后使用 Pyecharts 库绘制了中国地图，并在地图上标注了各城市的地铁线路数量。使用 EffectScatter 类型来展示数据点，并设置颜色范围从浅红到深红，通过颜色的深浅变化，可以清晰地看到不同城市的地铁线路数量差异。

```

# 计算最大值和最小值
max_count = max(line_counts)
min_count = min(line_counts)

# 绘制地图
geo = (
    Geo()
    .add_schema(maptype="china")
    .add(
        "",
        [list(z) for z in zip(cities, line_counts)],
        type_=ChartType.EFFECT_SCATTER,
    )
    .set_series_opts(label_opts=opts.LabelOpts(is_show=False))
    .set_global_opts(
        visualmap_opts=opts.VisualMapOpts(
            min_=float(min_count),
            max_=float(max_count),

```

从本次地铁线路数据的可视化结果来看，北京在地铁线路建设方面处于领先地位，其地铁线路数量多达 26 条。这充分体现了北京作为全国政治、经济、文化中心，人口密集、城市规模庞大，对于公共交通的庞大需求以及在城市轨道交通建设方面的雄厚实力和长期投入。众多地铁线路的布局，不仅为市民提供了便捷的出行选择，也有力地促进了城市不同区域之间的互联互通，推动了城市经济、文化等多方面的交流与发展。

相比之下，深圳的地铁线路数量相对较少，目前仅有 14 条。尽管深圳是一座充满活力的现代化大都市，经济发展迅速，但由于其城市发展历程相对较短，地铁建设起步也稍晚于北京等城市。不过，深圳的地铁建设速度一直很快，近年来也在不断加速线路的规划与建设，未来有望进一步提升地铁线路的数量和覆盖范围，以更好地满足城市日益增长的交通需求，进一步提升城市的交通便利性和综合竞争力。

各城市地铁线路数量分布




```
fig, ax = plt.subplots(figsize=(12, 12), subplot_kw={'projection':
ccrs.PlateCarree()})
ax.set_extent([120.95, 122.0, 30.9, 31.45], crs=ccrs.PlateCarree())
```

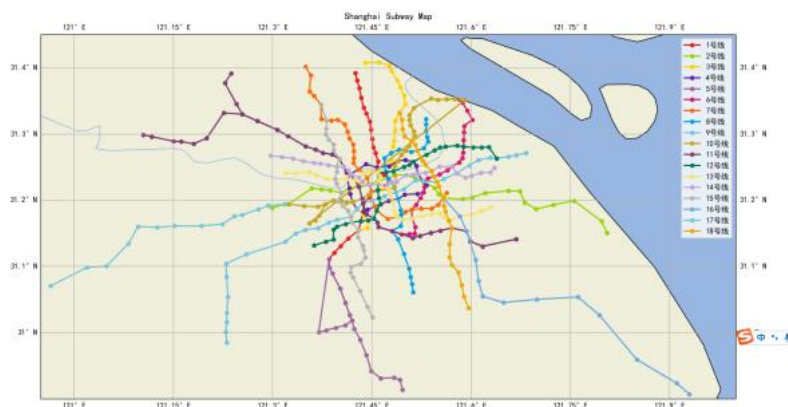
- 地理特征添加:

在地图上添加陆地、海洋、海岸线、国界线和河流等地理特征。

```
ax.add_feature(cfeature.LAND)
ax.add_feature(cfeature.OCEAN)
ax.add_feature(cfeature.COASTLINE)
ax.add_feature(cfeature.BORDERS, linestyle=':')
ax.add_feature(cfeature.RIVERS)
```

- 地铁线路绘制:

根据地铁线路名称，筛选出对应的站点经纬度数据，并绘制地铁线路图。但如果就这样直接完成代码，不加别的处理，会发现 5 号线和 10 号线存在错误连线，如下图所示。



所以对数据再次进行处理，将它们分别分割成两条独立的线路。最终代码部分如下：

```
line_colors = {
    '1号线': '#F00030', # 1号线正红色
    '2号线': '#90D800', # 2号线翠绿色
    # ...
    '5号线1': '#904890', # 5号线紫红色
    '10号线1': '#C0A8D8', # 10号线浅紫色
}

for line_name, color in line_colors.items():
    line_data = df[df.iloc[:, 8] == line_name]
    lons = line_data.iloc[:, 4] # 经度
    lats = line_data.iloc[:, 5] # 纬度
    if line_name not in ['5号线1', '10号线1']:
        ax.plot(lons, lats, marker='o', color=color, label=line_name,
markersize=3, linewidth=1)
    else:
        ax.plot(lons, lats, marker='o', color=color, markersize=3, linewidth=1)
```

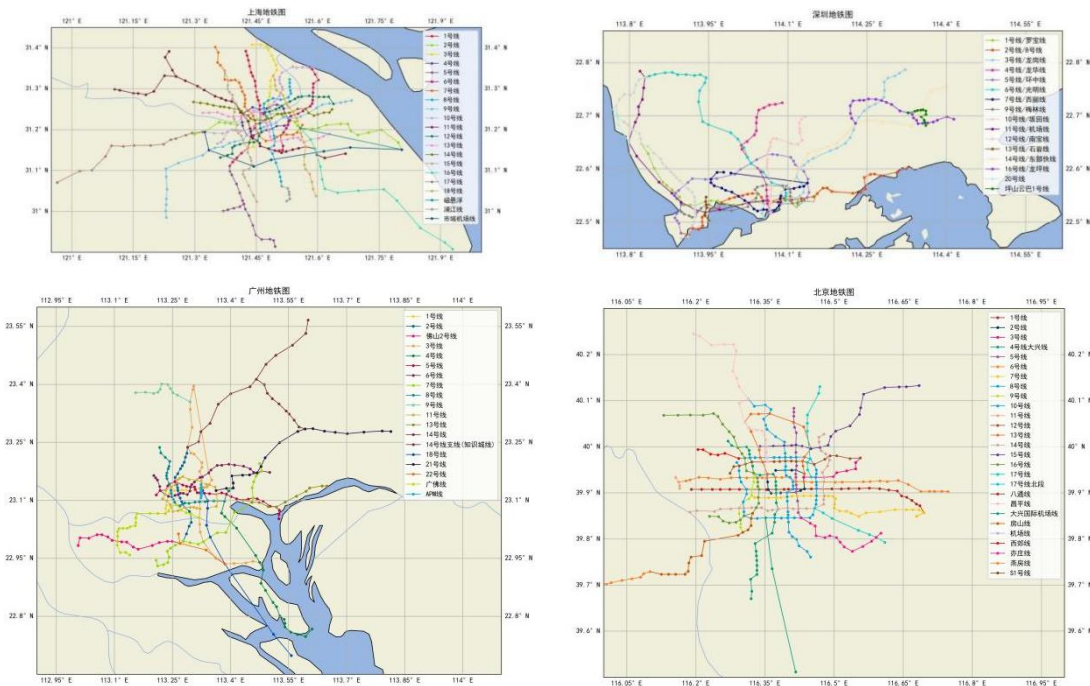
- 图例和网格线添加

在地图上添加图例和网格线，以便于识别地铁线路和站点位置。

```
ax.legend(loc='upper right')
ax.gridlines(draw_labels=True)
```

➤ 结果分析

通过上述步骤，我们能够成功绘制北上广深四个城市的**地铁线路图**。图中清晰地展示了各条地铁线路的分布情况，以及各个站点的位置，对比分析如下：



● 地铁线路数量和覆盖范围：

上海：从图中可以看出，上海地铁线路数量较多，覆盖范围广，几乎覆盖了整个市区，且线路分布较为均匀，这对于市民的日常出行提供了极大的便利。

深圳：深圳地铁线路数量相对较少，但覆盖了主要的市区和一些重要的区域，对于核心区域的出行需求基本可以满足。

广州：广州地铁线路数量适中，覆盖范围也较为广泛，特别是在市区内，线路分布较为密集，能够有效满足市民的出行需求。

北京：北京地铁线路数量多，覆盖范围广，不仅覆盖了市区，还延伸到了郊区，线路分布较为均衡，对于市民的出行提供了良好的服务。

● 站点密度：

上海：站点密度高，特别是在市中心区域，站点之间的距离较短，方便市民快速到达目的地。

深圳：站点密度相对较低，但在主要商业区和居住区，站点分布较为合理。

广州：站点密度较高，特别是在市区内，站点之间的距离较短，出行便利。

北京：站点密度高，特别是在市中心区域，站点分布密集，能够有效减少市民的步行距离。

● 换乘便利性：

上海：换乘站点较多，换乘便利性较好，但部分站点可能存在换乘距离较长的问题。

深圳：换乘站点相对较少，换乘便利性一般，需要进一步优化换乘设计。

广州：换乘站点较多，换乘便利性较好，能够有效减少市民的换乘时间。

北京：换乘站点多，换乘便利性较好，但部分站点可能存在换乘复杂度较高的问题。

● 综合评价：

综合考虑地铁线路数量、覆盖范围、站点密度和换乘便利性等因素，可以得出以下结论：

北京和上海在地铁出行便捷程度上表现最佳，线路数量多，覆盖范围广，站点密度高，换乘便利性较好，能够有效满足市民的出行需求；广州的地铁出行便捷程度也较高，线路数量适中，覆盖范围广，站点密度高，换乘便利性较好；深圳的地铁出行便捷程度相对较低，虽然覆盖了主要区域，但线路数量和站点密度相对较低，换乘便利性一般，需要进一步提升地铁网络的覆盖和服务能力。

4.2 城市道路拥挤情况分析

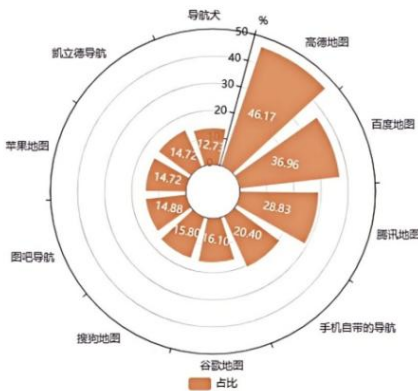
在城市交通服务水平的测度方面，通常采用交通运行指数进行评价，国内外围绕这一概念提出的指标及算法有数十种之多，例如国外的道路拥堵指数(RCI)、拥堵持续指标(LKDI)、拥堵度(DC)、出行时间指数(TTI)、拥堵严重度指标(CSI)、拥堵指数(CI)等，以及国内常用的交通运行指数(TPI)等。这些指标无一例外需要大量实测数据的支持方能计算，由于数据获取门槛的存在使得其仅能为专业机构所用，不便于普及。并且其中部分指标是基于半天或全天的累计交通数据进行的计算，对于需要把握实时交通变化的情况适用性不高。以百度地图和高德地图为代表的国内在线电子地图通过采集用户数据以及专业浮动车数据相结合陆续推出了各自的实时路况服务，虽然这些互联网公司并未全部公开其计算方法和分级标准，但其提供的实时动态数据无疑为城市交通服务水平的评价提供了一个新选择。

4.2.1 数据获取

(1) 数据源的选择

国内在线电子地图供应商主要有三家，即百度地图、高德地图和腾讯地图。在艾媒报告的 2024 年中国手机地图市场研究报告中指出，高德地图市场份额已达 46.17%，百度地图以 36.96%的份额居于次席，腾讯地图仅占有 28.83%。

2024年中国网民地图导航APP使用情况



市场份额近似等同于用户数量，换言之，较多的用户基数有利于供应商搜集更多的用户反馈信息，供应商提供的地图信息也相对更精确。三家在线电子地图的主要特色也各不相同，百度地图通过收购长地万方获得了甲级测绘资质，并在

境外积极与诺基亚的 HERE 地图展开合作，在国内外地图覆盖率方面具有显著优势。高德地图的强项在于导航，高德不仅具有与四维图新并驾齐驱的测绘实力，并且拥有从数据加工到终端平台的完整生态链，除了手机地图市场外，高德在车载导航领域也处于领军地位，积累了大量轨迹数据用于修正路况信息。腾讯地图则以街景服务见长，其前身搜搜地图是国内最早上线街景数据的供应商，其独特的街景时光机功能可以为城市空间纵向变化对比提供支持。

从应用程序编程接口（API）方面进行比较，以高德地图为例，虽然其开发指南中提供了交通态势的查询功能，但其口径是矩形或圆形区域，返回的结果是该区域的整体水平与研究所需的路段口径不一致，因此需要针对其实时路况图层逐个街道提取信息。为此，需要将地图中不必要的元素予以剔除，即在线电子地图的“自定义”功能。百度地图的“自定义”功能最为完善，允许用户对预先设定好的 4 个大类、12 个子类地图元素进行控制，并提供了 2 个属性、4 个子属性来调整各类元素的样式与显隐。高德地图此方面相对薄弱，仅提供了 5 中预设的地图样式，以及 4 个大类地图元素的显隐控制。腾讯地图则完全没有提供此类功能，给实时路况信息的提取造成了较大的阻碍。

三家供应商的实时路况数据图层也略有差异，高德地图和百度地图的实时路况既包括实时数据也可以查询预测数据，高德地图的实时路况信息相对纯净（图 X），百度地图的实时路况则与兴趣点数据绑定在一起，部分路段被文字压盖（图 X），不便于处理。腾讯地图的实时路况仅提供实时数据，并未提供预测数据，可挖掘的潜力最低。



图 X 高德地图实时路况



图 X 百度地图实时路况

综合地图特色、API 自定义功能、实时路况数据的品质三方面，首先高德地图本身见长于提供导航服务，在实时路况的计算方面已有足够的优势；其次，高德地图 API 提供了必要的自定义功能控制地图元素的显隐，可以支持对实时路况图层的单独提取；最后，高德地图的实时路况数据图层信息最为简洁，处理难度较低。

（2）实时路况数据的获取

选定合适的数据源后，需确定数据获取的整个流程框架。获取实时交通状况共分 4 个步骤：第一步用编程语言访问高德地图 API，截取多个时刻的路况图片；第二步应用地理信息系统软件对所得栅格数据进行加工，包括图片通道分离和重分类等操作；第三步对路网进行矢量化，计算出各路段中点；第四步利用路段中点提取对应位置的栅格属性。

➤ 实时交通状况图片的截取

大数据分析中最为常用的语言当数 R 和 Python，前者偏重数学计算，后者偏重工程应用。Python 中有大量编写好的库可供调用，其中用于浏览器应用程

序测试的 Selenium 可以对浏览器窗口尺寸控制、HTML 文件访问、窗口内容截取等一系列操作进行自动化处理，以实现大尺寸图片的截取，在合适的缩放级别下，单张图片即可覆盖城市建成区的大部分面积。以微软 Edge 浏览器为例，涉及的主要代码思路如下：

- 初始化浏览器配置：

定义访问网页所需的 Cookie 信息和请求头，从而模拟用户登录状态下浏览器的请求，避免网站页面反爬。

```
cookie = 'uuid_n_v=v1;
uuid=48D3B15024BC11F09332E374B54F4831A95B62F3F82B4769933E53B18F09923B;
_csrf=93e4b40fc4bbb96e55ffff4f89e0faa0e166e2ca9411ef76b05d28e80b683895'
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36 Edg/135.0.0.0',
    'Cookie': cookie,
    'Accept-Language': 'zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6',
}
```

- 设置浏览器选项：

首先创建 Edge 浏览器的配置选项，同时通过设置无头模式，使浏览器在后台运行，不显示窗口。这对于自动化任务非常有用，尤其是在服务器环境中。对于本项目而言，由于需要 24 小时不间断完成 HTML 内容的截取，不显示窗口才能不影响正常电脑设备的使用，是很有必要的。

```
from selenium import webdriver

# 设置浏览器选项，使窗口在后台运行不显示
opt = webdriver.EdgeOptions()
opt.add_argument('--headless') # 使用无头模式
```

- 设置窗口大小：

设置浏览器窗口大小为 2000x2000 像素，这确保了截图能够完整地捕获页面内容。此外，由于每张截图的大小且位置均一致，也能方便后续可视化处理时的循环自动化操作。不然需要对每张截图识别道路分布，再提取成点，上千张图片的手动处理几乎是不可能的。

```
# 设置窗口大小
driver.set_window_size(2000, 2000)
```

- 定时截图逻辑：

首先需要初始化一个计数器 c，用于记录截图次数。随后使用 driver.get() 打开本地 HTML 文件，使用 time.sleep(10) 等待 10 秒，确保页面完全加载完成。当初设置时认为 10 秒已经足够久了，初步测试也没有出现问题。但后来完成 24 小时的全部截图后，每个城市还是都出现了十张以内页面加载不完全的情况。鉴于时间有限，且页面加载不完全的情况占比极低，最终决定不再调整参数重新截图，而是将加载不完全的页面截图作为异常数据，后将用其前后两张截图对应道路拥挤度的平均值替代。

- 截图文件保存:

通过**获取当前时间戳来命名截图文件**，好处是后续图片数量较多时，方便对数据的直接观察和批量化操作，也能避免可视化过程中文件命名不规范导致报错的情况。这里选择每次截图后**等待 280 秒**，是因为每次打开 HTML 文件后，会有 10 秒的等待时间确保页面加载完成。相当于此处选择**每隔 5 分钟截图一次**，从而检测各个城市道路拥挤程度的实时变化。

```
# 获取当前时间戳，用于截图文件名
timemark = time.strftime('%Y_%m_%d_%H_%M_%S')

# 保存截图
driver.save_screenshot(f"C:/Users/ThinkPad/Desktop/路网/上海/{timemark}.png")
print(f"截图已保存: {timemark}.png")

# 清空当前页面内容，避免内存泄漏
driver.get('about:blank')

# 计数器加1
c += 1

# 每次截图后等待 280 秒
time.sleep(280)

import time

while True:
    # 打开本地 HTML 文件

    driver.get('file:///C:/Users/ThinkPad/Desktop/%E8%B7%AF%E7%BD%91/shanghai

    # 等待页面加载完成
    time.sleep(10)
```

- 考虑因素:

以上代码设计是由于动态数据爬取的过程中遇到了困难，各个线上地图平台的**反爬措施都很完善**，因此除了**使用其 API 接口**，几乎无可能性直接获得大量的数据。选择每隔 5 分钟截图的原因是 API 接口**有每日使用上限的要求**，无法支持更精细的数据采集。原本有考虑直接爬取动态数据，获得拥挤程度的数字指标，但正如上文有提及的，**很多指标都是基于半天或全天的累计交通数据进行的计算**，无法直接通过其数值分析得出城市通勤的便利度。因此选择对 API 的实时路况图层进行处理，隐去所有无关信息，然后通过静态图片的频繁截取自行分析获得数据。由于高德地图的实时路况图层运用了不同的颜色表达不同的拥挤状态，因此可以通过提取颜色通道信息，然后将截图中的路段尽可能**平均且密度较高的进行切分**，提取每小段的颜色信息，**通过矢量化转成拥堵信息**，用 0~4 代替，随后对每张截图中的拥堵信息进行**求和以及计算占比**，最终完成城市道路**实时拥挤程度变化的可视化**。

- HTML 文件的设计:

上文代码指向预先编写好的 HTML 文件，而 HTML 文件基于高德地图 API 的实时路况图层，因此将在高德低的 API 的实时路况图层代码的基础上，直接进行修改。

通过观察和尝试，可以发现以下这段代码内容用来控制页面右下角，有一个 显示/隐藏实时路况的按钮，但如果截图，这个按钮就会挡住一部分的道路，使得信息提取不安全。因此直接将这段代码删去。

```
<div class="input-card" style="width:
auto;">
  <div class="input-item">
    <button class="btn" id="btn">显示/
隐藏实时路况</button>
  </div>
</div>
```

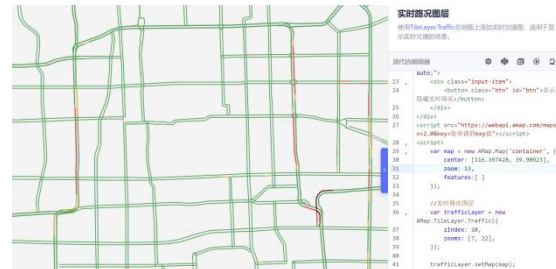
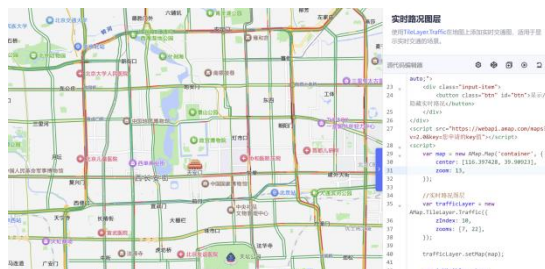


将申请获得的 开发者 key 代入代码，方便代码保存后的本地直接运行。

```
<script src="https://webapi.amap.com/maps?
v=2.0&key=ffe9cdf51de8f2fea67d01ac065e3970"></script>
```

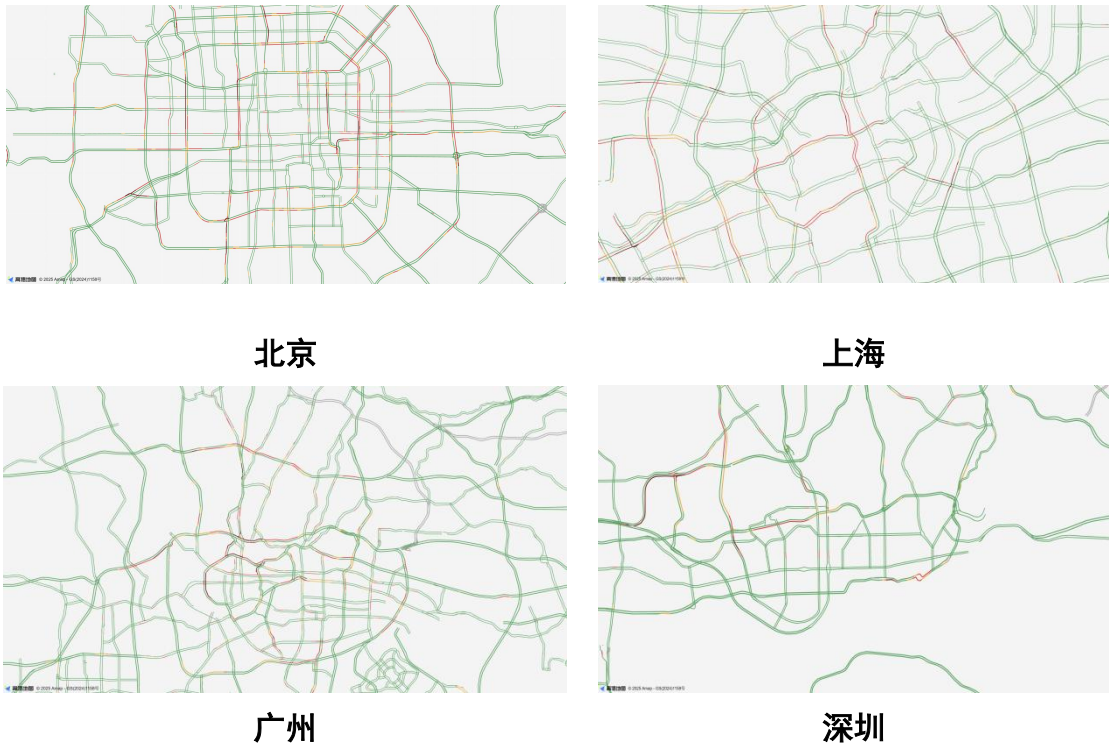
随后，将地图风格设置为深色、仅显示建筑物，利用建筑物图层仅在 17、18 缩放级别显示的特性，在截取实时交通路况时将缩放级别控制在 16 级及以下即可隐去建筑物，得到纯净的地图背景。在这里 缩放级别控制 在 12 到 13 级之间，方便完整截取整个城市的拥挤信息。然后，再加入 features: [] 以 隐藏建筑名、路名、地理特征 等，便于后续矢量化操作。

```
var map = new AMap.Map('container', {
  center: [121.47, 31.22],
  zoom: 12.7,
  features: []
});
// 实时路况图层
var trafficLayer = new AMap.TileLayer.Traffic({
  zIndex: 10,
  zooms: [7, 22],
});
```



最后，将完全修改好且确认可以运行的 API 代码，保存到本地的 txt 文件 中，

再将后缀直接改为 **html**，将代码指向该文件，便可以完成实时交通状况图片的截取。最终四个城市的实施交通状况图片如下：



➤ 归纳不同状态颜色的 RGB 组合

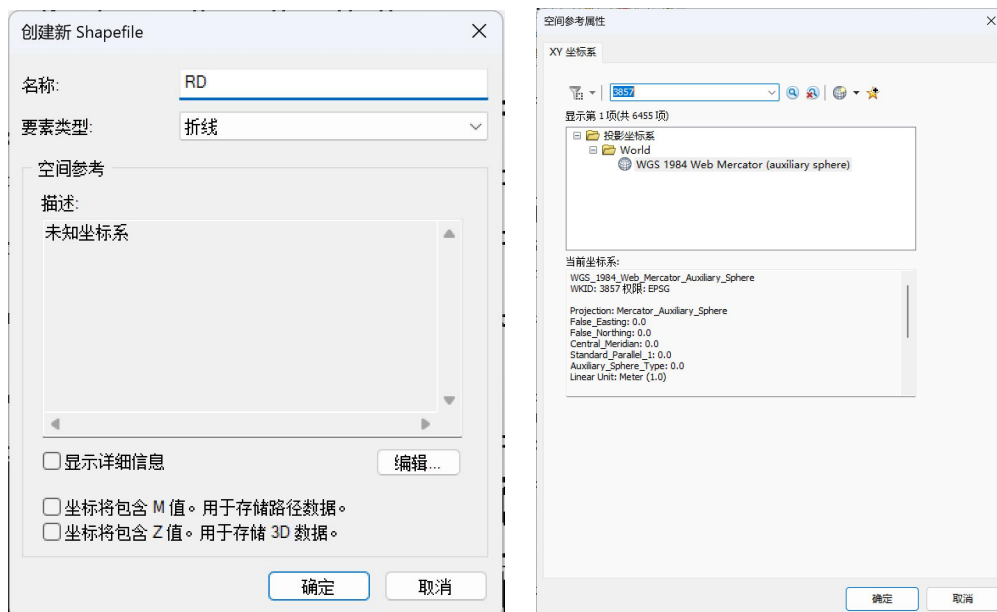
通过 Python 调用浏览器截取图片为 PNG 格式，图片中含有红色(Red)、绿色(Green)蓝色(Blue)三个通道，或称为三个波段。高德地图对交通拥堵共定义了 4 个级别，且在地图中用灰色标明了无数据的路段，利用绘图软件查看这 5 种路况对应的通道组合。具体操作为：打开绘图软件（此处使用 Photoshop），加载截取的 PNG 图片；然后使用绘图软件的颜色拾取工具，分别点击地图上不同路况的颜色区域（如绿色、黄色、红色、暗红色、灰色和背景色），查看并记录每种颜色对应的 RGB 值。

将记录的 RGB 值整理成表格，如下所示：

表 X 高德地图实时路况颜色释义与通道组合				
颜色	释义	Red	Green	Blue
绿	畅行	98	166	105
黄	缓行	230	174	68
红	拥堵	215	75	75
暗红	严重拥堵	112	33	33
灰	无数据	173	173	173
背景 1	地图背景	244	244	244
背景 2	地图背景	247	247	247

➤ 进行矢量化，得到中心线

在矢量化操作前，**创建折线要素**以记录每条路段的拥挤信息是有必要的，具体操作如下。先创建 city_SHP 的文件夹，用来存储所有矢量化有关的文件，方便后续查找。直接在 ArcMap 中，文件目录下新建一个 shapefile，选择折线类型，**命名为 RD 以代表城市道路**。随后点击编辑，选择空间参考属性。虽然我们要操作的都是截图文件，后续的处理单位也都应该是像素格，但可以在这里强行将坐标投影到**自然地理**的坐标，方便后续操作。



同理在相同的文件夹下**创建面文件**，命名为 Error。在这里创建面文件主要是因为通过观察，可以看到截图文件的左下角有高德地图的水印，猜测可能用折线提取时，会出现报错，因此为了**避免出现无法识别的信息系统报错**，创建面文件以防万一。

随后，**进行矢量化操作，提取中心线**。

● 设置 ArcScan 工具：

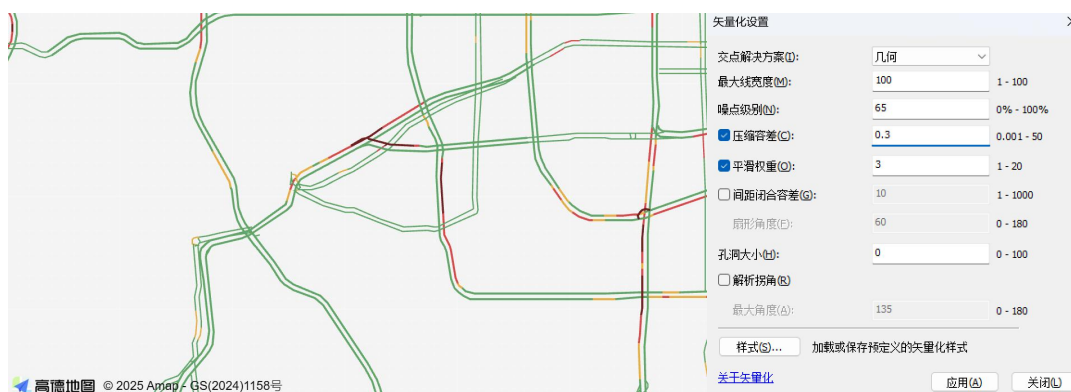
打开 ArcScan 工具栏

如果 ArcScan 工具栏没有显示，可以通过自定义 > 工具栏 > ArcScan 来启用。

● 设置矢量化参数：

在 ArcScan 工具栏中，点击“设置”按钮，打开“矢量化设置”对话框。

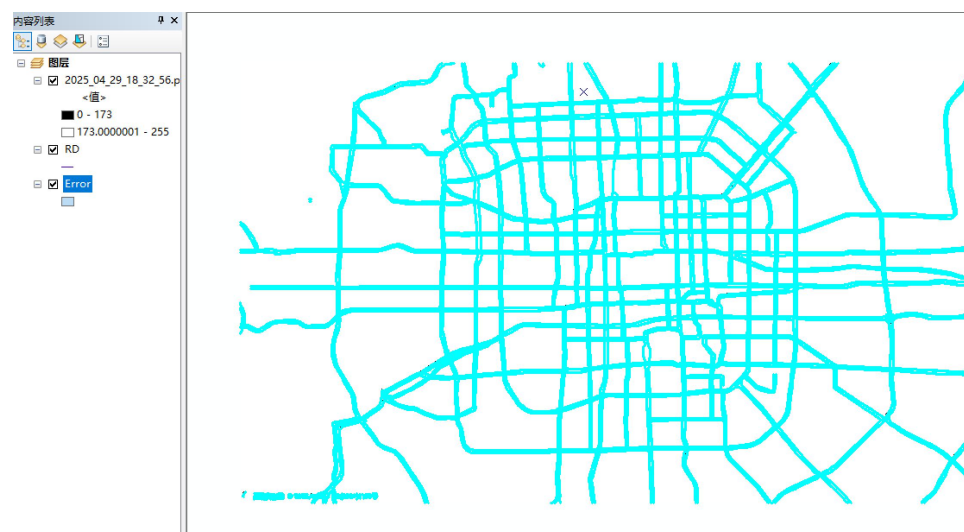
在“矢量化设置”对话框中，将以下默认参数进行修改：



最大线宽是指在矢量化过程中，软件识别为“线”的栅格宽度的最大值。如果栅格数据中的道路宽度超过这个值，软件可能会将道路识别为多个独立的线段，而不是一条连续的线。此处将最大线宽度将被**设为 100**，因为在交通地图中，道路的宽度可能会有很大差异，从狭窄的小巷到宽阔的高速公路。设置一个较大的最大线宽可以确保软件能够识别并矢量化各种宽度的道路，避免因道路过宽而被误识别为多个线段，从而**适应道路宽度变化**。此外，栅格数据的分辨率可能会影响道路的宽度表示。如果栅格分辨率较低，道路可能会被表示为较宽的区域。设置较大的最大线宽可以更好地**适应不同分辨率的栅格数据**。

压缩容差用于控制矢量化线要素的折点数量。较大的压缩容差会减少折点数量，使线更加平滑；较小的压缩容差会保留更多的折点，使线更加精确地反映原始栅格数据的形状。此处将压缩容差**设置为 0.3**，因为在矢量化过程中，需要在保留道路走向的精度和减少折点数量之间找到平衡。如果压缩容差设置过小，折点数量会过多，导致线要素过于复杂，不仅增加了数据量，还可能在后续处理中引入**不必要的噪声**。如果压缩容差设置过大，折点数量会过少，可能导致线要素无法准确反映道路的实际形状。此外，设置合适的压缩容差可以**确保折点连线不会超出道路的实际范围**。如果压缩容差过大，可能会导致折点连线偏离道路中心线，甚至超出道路边界，从而影响后续分析的准确性。

点击“确定”保存设置。最终的矢量化设置参数如下：



- 设置输出要素类：

Step1: 在 ArcScan 工具栏中，点击“输出”按钮，打开“输出设置”对话框。

Step2: 在“输出设置”对话框中，设置以下参数：

- ✓ 输出要素类：选择之前创建的**折线要素 (RD)**，同时让无法识别的数据输出保存至**面要素 (Error)**。
- ✓ 属性字段：根据需要，可以添加额外的属性字段，这里就不用添加了。

Step3: 点击“确定”保存设置。

- 执行矢量化操作：

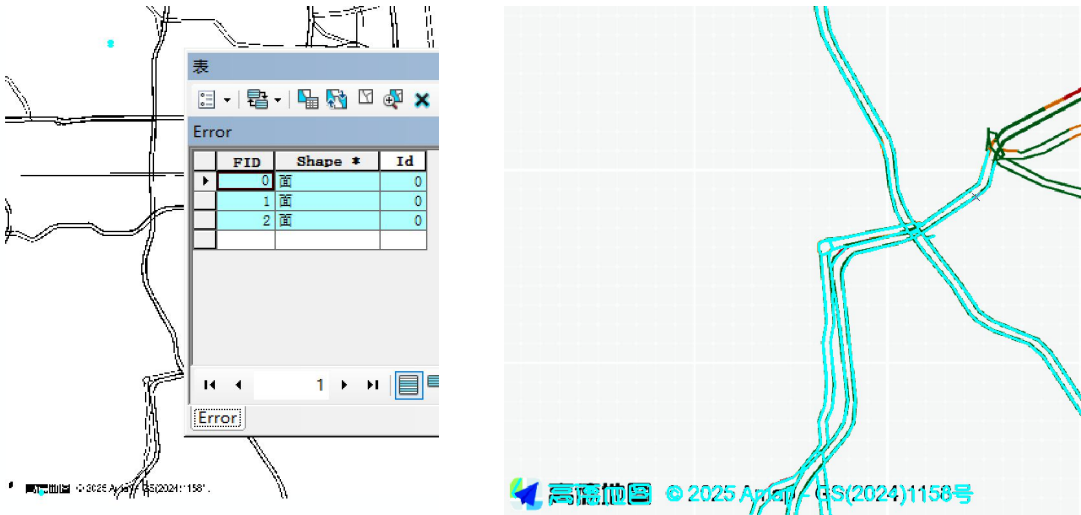
在 ArcScan 工具栏中，点击“开始矢量化”按钮。

ArcMap 将自动对栅格数据进行矢量化处理，将栅格图像中的道路信息转换为折线要素。

矢量化完成后，打开折线要素（RD）和面要素（Error）的**属性表**，检查是否正确提取了道路信息，打开编辑，全选所有道路线条，确认都被折线要素提取完毕后，直接保存。

● 清理无关信息：

首先，由于面要素是作为 Error 项的备选保存位置，所以打开面要素的属性表，如果期中均为**无关信息**（例如高德地图标识或其他背景区域），可以在 ArcMap 中**手动选择并删除**这些要素。如下图所示：



此外，叠加完整彩色的地图视图，可以看到左下角的**高德地图水印**，也被当做道路信息，提取到了折线要素中，同样**手动删除**这些无关线条，保存。至此，我们得到了一个修改后的 RD.shp 文件，包含了所有道路的矢量化信息，矢量化操作完毕。

➤ 对中心线进行增密，并转为中点

在地理信息系统（GIS）数据处理中，道路中心线的处理是关键步骤之一，尤其是在分析道路拥堵、交通流量等应用场景中。由于之前在数据处理过程中设置了压缩容差参数，该参数的作用是减少折点数量以简化道路几何形状，从而提高数据处理效率。然而，这种简化虽然有助于减少数据冗余，但也带来了新的问题：如果直接在剩余的折点处拆分道路，会**导致拆分后的道路段长度非常不均匀**。此外，每段道路可能包含**多个不同的拥堵状态**，这使得后续的数据处理变得复杂且难以准确分析。因此，为了优化道路数据，使其更适合后续分析，需要对中心线进行进一步处理。具体处理操作如下：

第一步：增密处理

由于之前设置的压缩容差参数使得折点尽可能的少，此时如果直接在折点处拆分道路会非常不均匀，且可能每段道路都包含多个拥堵状态，不利于后续的数据处理，因此在这里使用增密的工具。增密工具的作用是在**现有折点之间插入新的点，从而增加折点的密度**。具体操作步骤如下：

● 选择增密工具：

这个工具可以按照指定的距离或数量在现有折点之间插入新的点。

● 设置增密参数：

在增密工具的参数设置中，选择“按距离”进行增密。

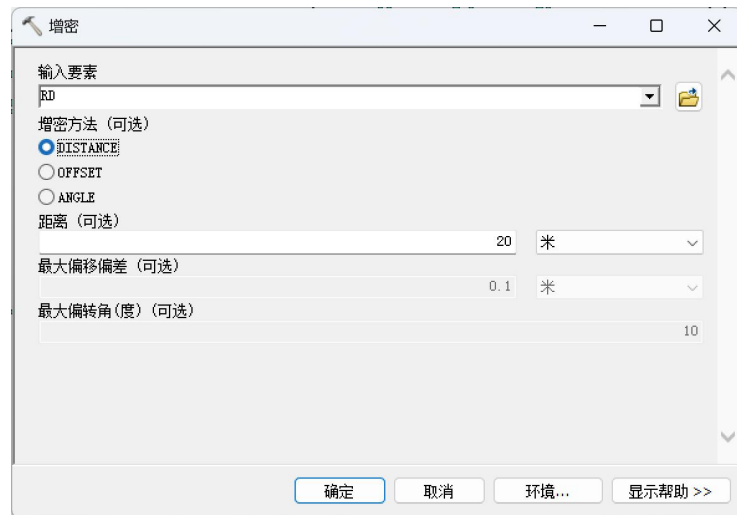
由于在创建折线文件时，选择了**自然地理**的投影坐标，因此可以将原来以像素为单位，转换成以米为单位。在这里，我们将**增密距离设置为 20 米**。这意味

着在每两个现有折点之间，系统会自动插入新的点，使得任意两个相邻折点之间的距离不超过 20 米。这样的设置可以确保道路数据的均匀性，同时避免因折点过少而导致的分析误差。

- 执行增密操作

设置好参数后，点击“确定”按钮，开始对中心线进行增密。

增密完成后，道路中心线的折点数量会显著增加，但道路的整体几何形状保持不变。



第二步：分割线

增密完成后，下一步是将道路中心线分割为更小的线段，以便后续分析。具体操作如下：

- 选择分割工具：

在 ArcMap 的工具栏中，选择“在折点处分割线”功能。

这个工具可以根据现有的折点将线要素分割为多个独立的线段，也就再次体现了上一步增密的意义所在。如果不进行增密操作的话，我们同样可以执行在折点处分割线的操作，但每条线段可能对应很长一段路，其中同时包含多个不同的拥堵状态，从而影响后续城市实施道路拥堵程度的计算。

- 执行分割操作：

选择已经增密后的道路中心线，然后点击“确定”按钮。

系统会根据增密后的新折点将道路分割为多个长度不超过 20 米的线段。将分割后的线段存储为一个新的要素类，命名为 Rd_SplitLine.shp。这样，每一段道路都具有相对均匀的长度，且每段道路的拥堵状态更易于分析。

第三步：折点转点

分割完成后，为了进一步提取道路信息，需要将分割后的线段的中点提取出来。具体操作如下：

- 选择折点转点工具：

在 ArcMap 的工具栏中，选择“要素折点转点”功能。这个工具可以将线要素的折点提取出来并生成新的点要素。

- 设置提取参数：

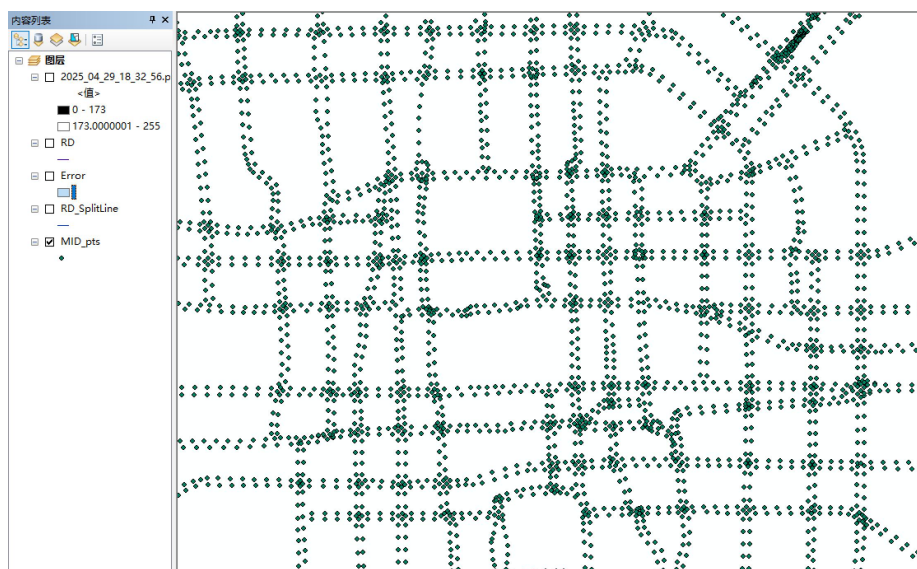
在折点转点工具的参数设置中，选择“中点”选项。这意味着系统会提取每一段线段的中点，而不是线段的起点或终点。

- 执行提取操作：

输入要素选择道路折线数据，即 Rd_SplitLine.shp 要素。

输出要素类型选择中点，执行工具，生成代表道路中点的点要素。

提取完成后，系统会生成一个新的点要素类，命名为“MID_pts”。这些中点可以用于后续提取栅格值等分析操作。



通过上述步骤，完成了对道路中心线的增密、分割以及中点提取的处理。这些操作使得道路数据更加均匀、准确，为后续的数据处理和分析提供了良好的基础。

➤ 利用 Python 对栅格数据拆分通道（R、G、B）

以上均为处理单张图片的实验性操作，在确认矢量化、中心线提取、增密分割和提取中点的操作可行后，以下将开始以城市为单位，对能反映其路况的截图进行批量化操作。由于记录时长共 24 小时，每章截图间隔时间为 5 分钟，因此理论上每个城市会有 288 张截图需要处理。事实上，每个城市的截图数量均为 300 张左右，原因是记录总时长实际略超过 24 小时，以防头尾数据记录不全。

首先，要批量化从指定的文件夹中读取所有 PNG 格式的图像文件，对每个图像进行通道拆分，提取出蓝色通道，并将其以简化的文件名保存到另一个指定的文件夹中。简化的文件名通过特定的规则生成，以确保符合 ArcGIS 对字段名称的限制（字段名不能超过 9 个字符且不能是纯数字）。通过这种方式，代码能够高效地批量处理图像数据，为后续的拥堵信息提取批量化提供经过预处理的蓝色通道图像文件。

```
# 定义原始栅格图像所在的路径
root = 'C:\\Users\\ThinkPad\\Desktop\\路网\\深圳\\'
# 定义处理后蓝色通道图像保存的路径
out = 'C:\\Users\\ThinkPad\\Desktop\\路网\\shenzhen_blue\\'

# 获取原始路径下所有文件的名称
pngs = os.listdir(root)
```

```

# 遍历文件列表
for p in pngs:
    # 检查文件是否为PNG格式
    if p[-4:] == '.png':
        # 打开图像文件
        image = Image.open(root + p)
        # 生成简化的文件名，符合ArcGIS字段名称限制
        short_name = 't' + p[5:-7].replace("_", ' ') + '.png'
        # 拆分图像通道
        r, g, b = image.split()
        # 保存蓝色通道到指定路径，使用简化后的文件名
        b.save(out + short_name)
    else:
        # 如果文件不是PNG格式，则跳过
        pass

```

➤ 重分类

重分类操作主要用于将栅格数据中的值重新分类为新的值，这种操作对于数据的标准化、简化分析过程以及满足特定的分析需求具有重要意义。

重分类操作的必要性：

- 标准化拥堵数据：

原始的交通数据包含多种颜色通道数值，但并不直观。

在实际操作的过程中，能发现其实各个颜色通道的 RGB 组合很难记住，即使只有蓝色通道，如上文提到的 105 为畅行、68 为缓行等，在操作过程中需要反复翻阅表格查看数据。同时这些数值在提取后，也难以直接通过 Excel 进行分析。通过重分类，可以将这些数值转换为统一的拥堵等级（如 1 至 4），从而便于进一步分析和可视化。

- 去除无关数据：

可以将与道路拥堵信息无关的栅格值设置为 0，避免对分析结果产生干扰。

由于数据的提取源为 HTML 的截图，因此每条道路的边缘部分都有由于柔化处理而包含无效信息。在这里无效信息的定义是，颜色通道数值不包含在上文列出的表格中，即不是四种拥堵状况中的任何一种，也不是无信息的灰色。我们可以认为这些栅格是无信息含量的，因此将其重分类标记为 0，以免在数值提取后，对分析结果产生干扰。

- 直观展示结果：

通过选择与实时路况地图类似的颜色标注，可以直观地展示拥堵情况。

重分类操作的步骤：

- 打开重分类工具，设置输入栅格数据：

在 ArcMap 的“ArcToolbox”中，找到“Spatial Analyst 工具”>“重分类”工具。在“重分类”工具对话框中，选择需要重分类的栅格数据作为输入。即加载任意一张上一步批量化获得的每页截图的蓝色通道，作为示例，完成重分类后，保存当前格式，方便后续模型构建器建模时，直接引入当前重分类方法。

- 定义重分类规则：

在“重分类”工具中，定义重分类规则。根据颜色通道数值，将拥堵状态分为以下等级：

- ✓ 无信息（实时路况地图显示灰色）：重分类为 0。
- ✓ **边缘柔化形成的无关栅格**：重分类为 0。
- ✓ 畅行（实时路况地图显示绿色）：重分类为 1。
- ✓ 缓行（实时路况地图显示黄色）：重分类为 2。
- ✓ 拥堵（实时路况地图显示红色）：重分类为 3。
- ✓ 严重拥堵（实时路况地图显示暗红色）：重分类为 4。

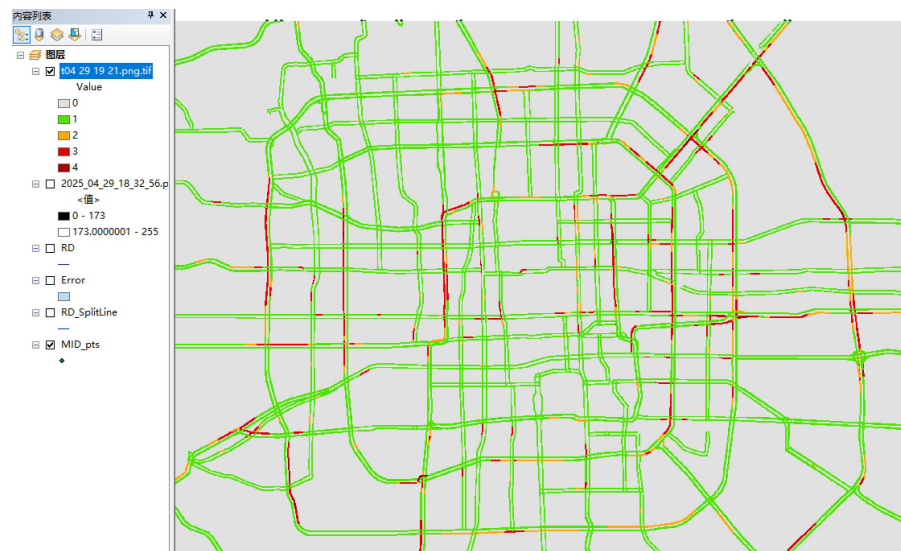
这些规则可以通过手动输入来定义。

- 设置输出栅格数据：

在“重分类”工具对话框中，指定输出栅格数据的保存路径和文件名。输出数据将是一个**新的栅格文件**，其值根据定义的规则进行了重新分类。

- 设置颜色标注：

加载输出的栅格数据到地图窗口中，在图层属性中，选择“符号系统”选项卡。选择“分类”符号系统，可以根据拥堵等级，**选择与实时路况地图类似的颜色**，最终可以直观地展示拥堵情况，如下图所示。



➤ 扩展操作

ArcMap 的扩展操作通常用于**增加栅格数据的有效区域面积**。这种操作对于提高数据的完整性和分析的准确性具有重要意义，尤其是在处理边缘柔化或部分数据缺失的图像时。

扩展操作的必要性

在处理栅格数据时，尤其是在从 HTML 文件中截图获得的图像中，常常会遇到**边缘柔化**的问题。这些区域可能包含无效值（在这里无效的定义同样是，颜色通道数值不包含在上文列出的表格中，即不是四种拥堵状况中的任何一种，也不是无信息的灰色），**导致有效数据的占比降低**。因此，可以通过扩展操作，试图增加有效数据的占比。

扩展操作的步骤

- 打开扩展工具，设置输入栅格数据：

在 ArcMap 的“ArcToolbox”中，找到“Spatial Analyst 工具”>“散度”工具。在“散度”工具对话框中，选择需要扩展的栅格数据作为输入。即加载任意一张上一步批量化获得的每页截图的**蓝色通道**，作为示例。这里只需要**验证扩展操作**

的可行性与必要性即可，结果也不需要保存，下文将通过模型构建器统一批量化操作。

● 定义扩展参数：

扩展值：指定用于填充无效区域的值。通常可以选择与周围有效数据最接近的值，在这里选择固定值 1 即可。

扩展范围：定义扩展操作的范围。在这里选择扩展到整个栅格范围。

● 设置输出栅格数据，执行扩展操作：

由于这只是一个示例，因此输出文件的命名可以随意。在这里把扩展前的输出命名为 test1，扩展操作后输出命名为 test2。可以验证 test2 的输出数据是一个新的栅格文件，其中大部分的无效区域已被扩展为有效值。

行标签	计数项:RASTERVALU				
105	6656				
75	345				
68	305		6656		
173	177		345		
140	110		305		
175	59		177		
244	19		15		
209	17				
25	15		7498	7850	95.52%
91	12				

行标签	计数项:RASTERVALU				
105	6923		6923		
75	364		364		
68	333		333		
173	206		206		
33	17		17		
244	5				
25	1		7843	7850	99.91%
29	1				
(空白)					
总计	7850				

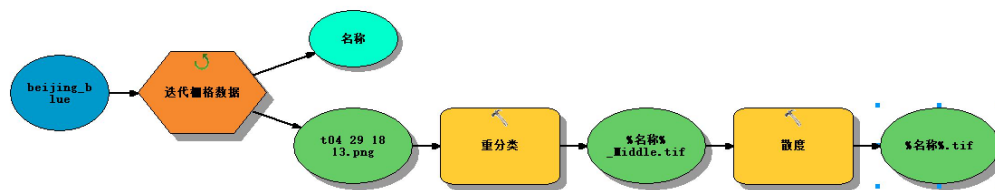
通过试验，在未进行扩展操作时，有效数值的占比仅为 95.52%。通过扩展操作，仅仅只是从中心像素格向外扩展 1 像素，也可以将大部分无效区域填充为有效值（毕竟无效区域的产生基本都是由于边缘柔化，因此无效区域围绕在道路中心线条周围），从而显著提高有效数据的占比，实际上扩展后也确实达到 99.91% 的有效数值。

综上，扩展操作不仅增加了数据的完整性，还减少了因边缘柔化或数据缺失对分析结果的影响，使得后续分析更加准确和可靠。

➤ 运用模型构建器，批量化操作重分类与扩展

模型构建器是一个在 ArcMap 中常用的工具，它用于自动化和简化地理数据处理流程。通过模型构建器，用户可以将一系列的地理数据处理步骤整合成一个模型，从而实现流程的自动化。这对于需要重复执行相同操作的场景特别有用，可以大大提高工作效率。在本研究中，主要将重分类和散度的操作写入模型构建器，便于批量化操作节省时间。具体操作过程中，每个城市的模型运行均耗时 1 小时左右，可见如果全手动操作，工作量过大几乎无法完成。在模型搭建的过程中，由于重分类后的文件被设置为了中间文件，因此右键增设“托管”，以保存中间文件的所有属性，以防运行过程中有误文件无法找回。在模型构建器中，重分类操作的分类方法可以直接引入之前试验时保存的分类文件，命名为 reclass，保存在 beijing_SHP 的文件夹中；散度操作同样是扩展 1 像素单位，与试验保持

一致。随后新建一个文件夹，命名为 city_blue_ExpRc，用来储存输出的所有文件。大致每个城市对应 1800 个文件，因此需要额外注意文件夹的整理和每个文件的规范命名。



➤ 多值提取至点，把栅格属性汇总到点要素上

多值提取至点是 ArcMap 中一个强大的空间分析工具，它用于将栅格数据集中的多个值提取到点要素的属性表中，也是点要素数据导出前的最后一个准备工作。

第一步：准备栅格数据和代表道路中点的点要素

这里需要确保所有栅格图像文件按照时间顺序命名，例如 t04_29_18_13.png，这样可以在后续分析中轻松识别每个栅格数据的时间戳。代表道路中点的点要素即为 MID_pts.shp，上文已有提及。

第二步：提取栅格属性到点要素

使用多值提取至点工具

输入点要素：选择上一步生成的点要素 MID_pts.shp。

在输入栅格数据中，添加所有需要提取的栅格图像文件，即将上文 2.5 节中批量化保存的所有截图的蓝色通道文件，添加到输入栅格数据中。

输出点要素指定保存路径和文件名，这里保存路径新建一个文件夹，保存到 city_blue_ExpRc。

执行工具，该工具会根据栅格数据的名称自动在点要素的属性表中创建字段，并将对应的栅格值附加到这些字段中。

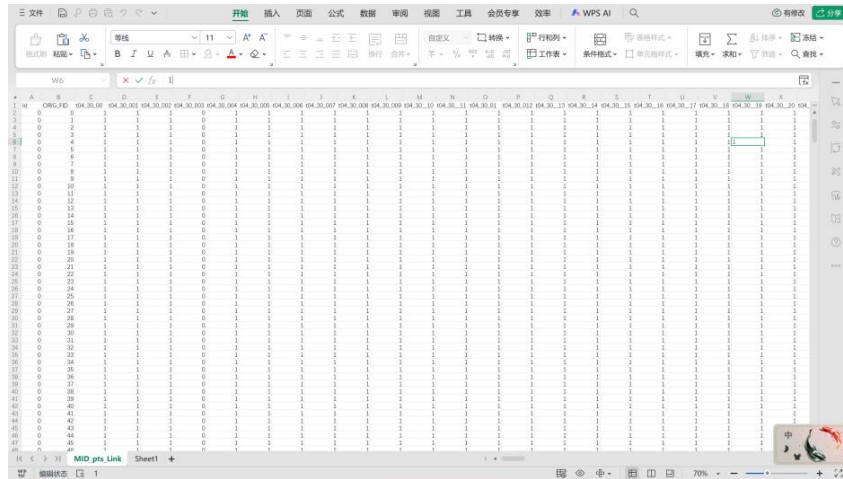
第三步：数据导出和统计分析

在 ArcMap 中加载包含拥堵信息的点要素文件，右键导出，导出到指定文件夹中，随后用 Excel 打开对应的 dbf 文件即可。

这里需要注意的是，通过试验发现，直接导出的文件中无法包含全部时刻的信息，但点要素的属性表里却是完全包含的。因此可以猜测是 Excel 有列数上限限制，或者导出时有最大列宽限制。最终解决方案是，将所有的截图文件，分成两部分使用多值提取至点，从而就能得到两张 Excel 表格，再手动检查一下时间范围，将两张表格合并。至此，所有数据获取的准备完成，可以开始可视化操作。

4.2.2 数据可视化

数据获取的所有操作步骤完成后，将会得到四个城市分别对应一个 Excel 文件。以下以广州为例，其 Excel 文件中共包含 8736 行，299 列。每行均对应一个分割后的路段的拥堵情况，分别用 0~4 的数值来量化，其中 0 为无信息，1 为畅行，4 为严重拥堵。可以预测，如果对每个时刻的道路拥堵情况做汇总统计，0、3、4 的数值会较少，而 1 最多。



为了得到各个城市拥挤情况的直观显示，编写程序对以上两百多万个数据进行处理统计。

- 初步处理，统计求和：

首先，分别统计每个时刻，拥挤度为 0~4 的路段总数；随后，计算拥挤度为 1~4 的路段总数之和，记为拥挤路段数；然后计算每个时刻得到的统计的路段总数，即拥挤度为 0~4 的路段总数；最终将每个时刻的拥挤路段数和路段总数相除，计算拥挤路段占比。以上操作均以列为单位，对每列分别进行计算，将所有计算结果按顺序记录在表格的结尾。

遍历每一列（时刻）

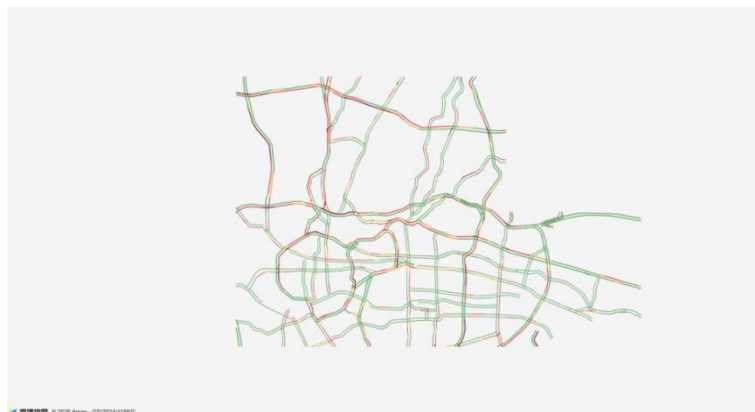
```
for index, col in enumerate(df.columns):
    counts = df[col].value_counts().to_dict() # 获取每个拥挤度的计数
    row = {
        '小时': index,
        '拥挤度': col,
        '0': counts.get(0, 0),
        '1': counts.get(1, 0),
        '2': counts.get(2, 0),
        '3': counts.get(3, 0),
        '4': counts.get(4, 0)
    }
    row['拥堵段落数量'] = row['2'] + row['3'] + row['4']
    row['总段落数量'] = row['0'] + row['1'] + row['2'] + row['3'] + row['4']
    row['拥堵比例'] = (row['拥堵段落数量'] / row['总段落数量']) * 100 \
        if row['总段落数量'] > 0 else 0
    stats_df = pd.concat([stats_df, pd.DataFrame([row])], ignore_index=True)
```

- 异常值判断与处理：

通过以上代码，将以上结果保存到 excel 文件中。通过观察可以看出，展示结果的数值与预期基本相符，除了高亮的单元格。

0	379	380	380	6932	381	381	380	380
1	8260	8248	8259	1756	8264	8273	8268	8272
2	34	39	33	24	28	30	33	28
3	45	45	49	16	46	37	39	41
4	17	23	14	7	16	14	15	14
	96	107	96	47	90	81	87	83
	8735	8735	8735	8735	8735	8735	8735	8735
	1.10%	1.22%	1.10%	0.54%	1.03%	0.93%	1.00%	0.95%

理论上记为 0 的路段数量应该是基本恒定的，因为 0 代表无路段拥堵信息，可能的原因是该路段正在修建，亦或暂不通车为仅供行人通行的小巷等。在展示的结果中，可以看出记为 0 的路段数量应该在 380 附近，但高亮的单元格显示记为 0 的路段数接近 7000，明显属于异常数据，所以高亮的单元格对应时刻一定出现了一些系统故障。返回截图文件中，根据相应的时间戳查找文件，可以发现该时刻对应的截图文件如下，很显然，这是由于截图时，浏览器页面仍未完全加载，导致的数据异常。



关于如何通过程序判断异常数据，这里选取的方法是先计算“拥挤度为 0 的路段总数”的整行数据的平均值。由于拥挤度为 0 代表无信息，因此上下波动应当较小，而即使异常数据会比正常值大十倍左右，但异常数据的占比极低，因此不会对平均值造成很大的影响。在这里认为，每列计数求和得出的“拥挤度为 0 的路段总数”，如果比先前计算出的整行平均值大，且差值超过 50，就将其判断为异常数据。

```
# 计算拥挤度为0的整行所有数据的平均值
avg_zero = stats_df['0'].mean()

# 标记异常值
wrong = []
for index, row in stats_df.iterrows():
    if abs(row['0'] - avg_zero) > 50:
        wrong.append(index)
```

而处理异常数据的方法是选取其上一时刻与下一时刻的对应数据，判断是否为异常数据。如果其左右列数据均不是异常数据，直接用平均值替代；如果其左右列数据中有异常数据，再判断其左右两列是否有异常数据，如果还有异常数据就判断左右三列，以此类推，最终用左右两个非异常数据的平均值替代。

```
# 替换异常值
for idx in wrong:
    # 获取当前行
    current_row = stats_df.iloc[idx]

    # 寻找非异常值行
    valid_rows = []
    for offset in [-1, 1]:
        place = idx + offset
        if 0 <= place < len(stats_df) and place not in wrong:
            valid_rows.append(stats_df.iloc[place])
```

```
# 如果找到非异常值行，计算平均值并替换
if valid_rows:
    replacement = pd.DataFrame(valid_rows).mean()
    stats_df.iloc[idx] = replacement
```

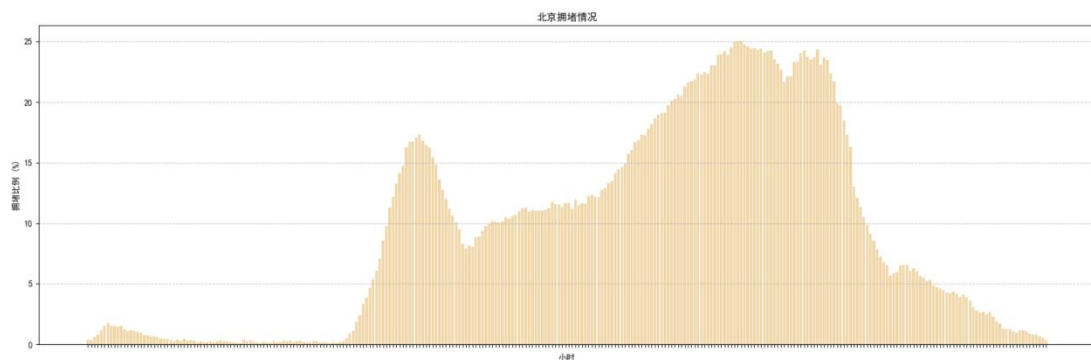
- 可视化操作：

由于每个数据点为间隔五分钟的实时拥堵情况，基于数据处理得到的不同时刻的拥堵路段占比，绘制柱状图。

```
# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei'] # 使用黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 绘制柱状图
plt.figure(figsize=(18, 6))
```

分别得到四个城市的全天实时路况变化图，纵坐标表示拥堵程度，用拥堵路段占总路段数的比例表示；横坐标表示时间，表示连续监控 24 小时。可视化结果与对应分析如下：



```
file_path = 'beijing_result.xlsx'
df = pd.read_excel(file_path)

valid_congestion_ratio = df['拥堵比例'][7859:-2]

hours_ticks = np.arange(0, 290, 1)
```



```
plt.bar(hours_ticks, valid_congestion_ratio, color='#FDD5A1')
plt.title('北京拥堵情况')
plt.xlabel('小时')
plt.ylabel('拥堵比例 (%)')
plt.xticks(ticks=hours_ticks, labels=[])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
plt.savefig('beijing.png', dpi=300)
```

从图中可以观察到，北上广深四个城市的交通拥堵均呈现出明显的**双高峰特征**。第一个高峰出现在早上7时左右，拥堵比例迅速上升，与早高峰时段的通勤需求相符。随后，拥堵比例逐渐下降，直到中午时段达到一个相对平稳的状态。第二个高峰出现在下午5时左右，拥堵比例再次迅速上升，与晚高峰时段的下班回家需求相符。之后，随着时间的推移，拥堵比例逐渐下降，直到深夜时段趋于平稳。



总体来看，这四个一线城市的交通拥堵主要集中在早晚高峰时段，这与城市居民的**通勤模式**密切相关。这种双高峰的拥堵模式提示城市规划和交通管理部门需要在这些时段采取相应的措施，如优化交通信号、增加公共交通运输力等，以缓解交通压力，提高城市交通的运行效率。



下文将针对每个城市的拥堵信息，做更进一步的分析。对每个城市的不同拥挤度的路段总数分别求和，然后计算其和有拥堵信息的路段总数的比值，从而得出不同拥挤度的路段数在总体的比重。

```

columns_0_to_4 = data[['0', '1', '2', '3', '4']]
sum_columns_0_to_4 = columns_0_to_4[7859:-2].sum()
sum_0 = sum_columns_0_to_4['0']
sum_1 = sum_columns_0_to_4['1']
sum_2 = sum_columns_0_to_4['2']
sum_3 = sum_columns_0_to_4['3']
sum_4 = sum_columns_0_to_4['4']

sum_all = sum_1 + sum_2 + sum_3 + sum_4

beijing_1 = sum_1/sum_all
beijing_2 = sum_2/sum_all
beijing_3 = sum_3/sum_all
beijing_4 = sum_4/sum_all

```

随后，通过绘制堆积柱状图，横向对比四个城市的拥挤程度。需要注意的是，在 python 实现过程中，堆积柱状图需要引入柱形高度的变量，通过计算每一段的底部高度，完成“堆积”的过程。

```

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei'] # 使用黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

cities = ['北京', '广州', '上海', '深圳']
traffic_data = {
    '畅行': [beijing_1, guangzhou_1, shanghai_1, shenzhen_1],
    '缓行': [beijing_2, guangzhou_2, shanghai_2, shenzhen_2],
    '拥堵': [beijing_3, guangzhou_3, shanghai_3, shenzhen_3],
    '严重拥堵': [beijing_4, guangzhou_4, shanghai_4, shenzhen_4]
}

# 准备数据
categories = list(traffic_data.keys())
values = np.array(list(traffic_data.values()))

# 设置颜色
colors = ['#4CAF50', '#FFC107', '#F44336', '#8B0000']

# 创建图表
fig, ax = plt.subplots(figsize=(6, 3))

```

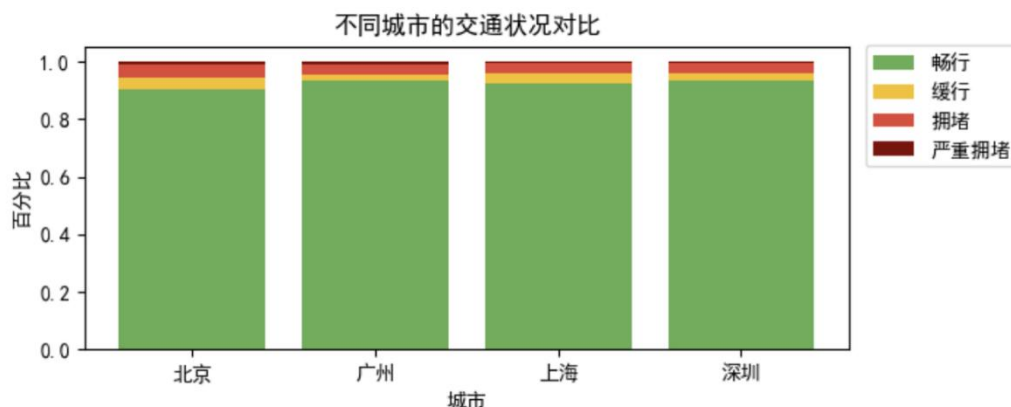
```

# 绘制堆积柱状图
bottom = np.zeros(len(cities)) # 初始底部高度为0
for i, category in enumerate(categories):
    bars = ax.bar(cities, values[i], bottom=bottom, label=category,\
                  color=colors[i])
    bottom += values[i] # 更新底部高度

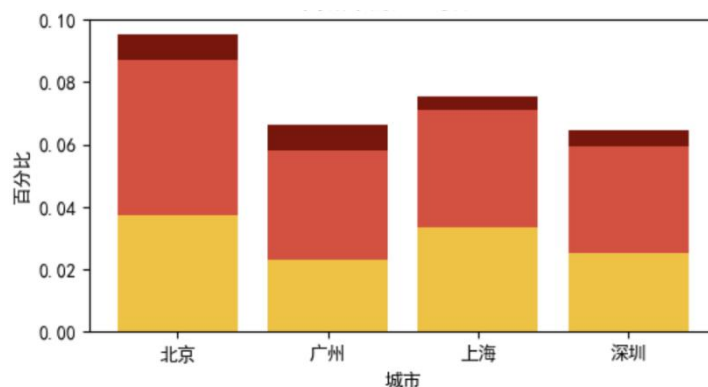
# 添加标题和标签
ax.set_xlabel('城市')
ax.set_ylabel('百分比')
ax.set_title('不同城市的交通状况对比')
fig.tight_layout()

```

得到不同城市的交通状况对比图如下，图中选取了与高德地图类似的颜色，方便直观表示畅行、缓行、拥堵和严重拥堵的概念。可以看出，在所有城市中，畅行的比例都相对较高，表明大部分时间内交通状况是相对顺畅的。其中，深圳的畅行比例似乎最高，这可能意味着深圳的交通管理较为有效或者道路条件较好。而拥堵的比例在四个城市中都占据了一定的份额，这可能与这些城市作为一线城市，人口密度高、车辆多有关。拥堵情况的存在可能会对市民的日常出行造成一定的影响。



但由于畅行比例在四个城市中占比都较高，很难直接通过图像横向对比四个城市的道路拥堵情况，因此**剔除畅行的数据**，仅保留拥堵的路段数据，再次绘制可视化图像，得到结果如下：



通过观察可以得出，北京和上海的整体拥堵情况更严重，这种差异可能与多个因素有关，包括城市规模、人口密度、车辆数量、道路容量以及公共交通系统的效率等。北京和上海作为中国的两个超大城市，拥有庞大的人口基数和密集的经济活动，这可能导致更高的交通需求和更频繁的拥堵现象。

但北京和广州严重拥堵的占比较高，甚至广州严重拥堵的路段占比超过了上海的严重拥堵路段。我认为原因可能有以下几点：首先，广州的交通状况可能受到特定区域如老商圈的老旧道路规划影响，这些区域车多人多而车道相对较少，导致拥堵问题较为严重。其次，广州的交通需求迅速攀升，尤其是随着人口的持续流入，交通承载能力不断受到挑战，尤其是在核心商业区、学校周边，交通量的激增让本就紧张的交通系统更加不堪重负。此外，广州的交通基础设施在面对不断增长的需求时，显得有些力不从心，大量的老旧路段和路网的瓶颈，仍然让交通流通性存在极大的短板。这些因素综合起来可能导致广州的严重拥堵路段比例高于上海。

综合而言，北上广深四个城市都有明显的拥堵情况，这可能需要这城市采取更有效的交通管理措施，如优化交通信号、发展公共交通、实施交通需求管理等。虽然广州和深圳整体而言的交通拥堵问题较轻，但对所有城市而言，为了防范未来交通情况的恶化，持续的交通管理和优化都是必要的，以提高交通效率和市民的出行体验。

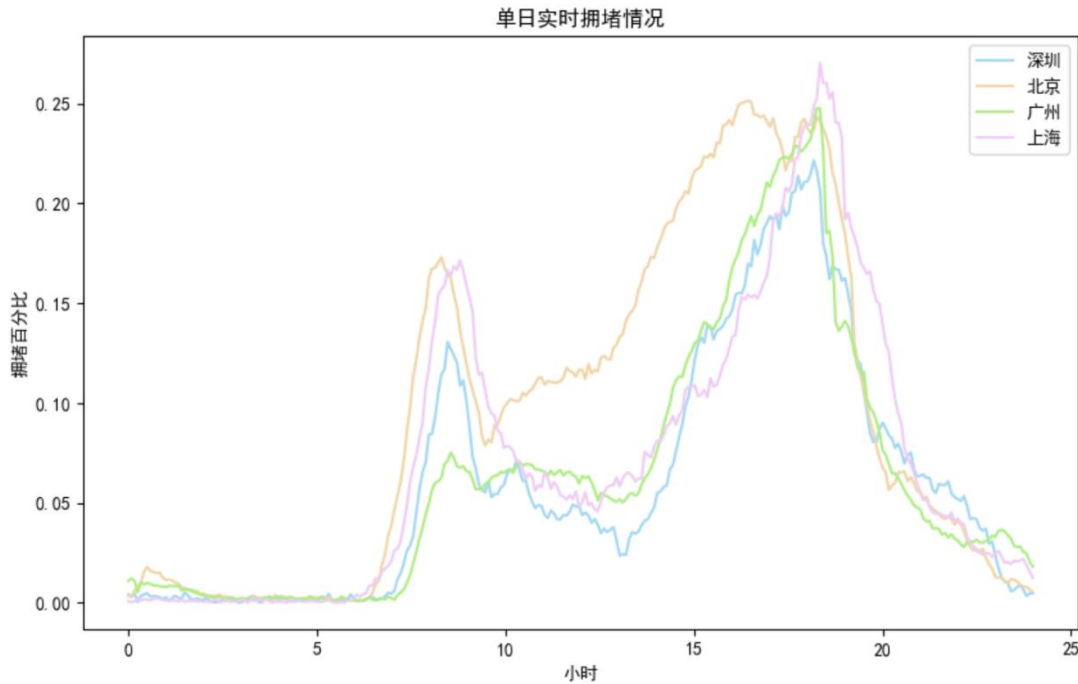
最后，综合比较各个城市的实时拥堵情况，用不同颜色的折线表示。

```
hours = data['时间']
shenzhen = data['深圳']
beijing = data['北京']
guangzhou = data['广州']
shanghai = data['上海']

# 创建图表
plt.figure(figsize=(10, 6))

# 绘制折线图
plt.plot(hours, shenzhen, label='深圳', color='#84D9F9')
plt.plot(hours, beijing, label='北京', color='#FDD5A1')
plt.plot(hours, guangzhou, label='广州', color='#8EF174')
plt.plot(hours, shanghai, label='上海', color='#FDCAFA')

# 添加标题和标签
plt.title('单日实时拥堵情况')
plt.xlabel('小时')
plt.ylabel('拥堵百分比')
```



通过对城市交通图像的分析，我们可以得出以下关于北京、上海、深圳和广州四个城市交通拥堵状况的详细结论：

● 上班时间的差异：

从图像中可以明显看出，北京是四个城市中上班时间最早的城市，这可能与北京作为国家政治中心，众多机关单位和企业集中，需要提前开始工作有关。上海紧随其后，其作为国际化大都市，金融、贸易等行业的工作节奏也较为紧凑。深圳作为新兴的科技和经济中心，上班时间稍晚于上海，但仍然早于广州。广州的上班时间最晚，这或许与当地的经济结构和生活节奏有关，其商业活动和工作安排相对更为灵活。

● 上下班高峰的拥堵特点：

上海和深圳的交通拥堵呈现出明显的“双峰”特征，即上下班高峰期间拥堵程度达到峰值，而中午时段拥堵状况明显回落。这种模式反映了这两个城市的工作和生活节奏较为规律，人们集中上下班，交通流量也随之变化。相比之下，北京和广州的拥堵情况则有所不同。北京从刚过中午开始，交通拥堵度就缓慢增加，这可能是由于北京的城市规模庞大，交通流量复杂，除了上下班高峰外，其他时段也有大量的交通需求。深圳虽然也有类似的趋势，但整体拥堵程度相对较低，这可能得益于其较为完善的交通规划和基础设施建设。

● 早高峰的拥堵峰值对比：

在早高峰时段，北京的拥堵峰值为 17.31%，与上海的 17.14% 非常接近，这表明这两个城市在早高峰期间的交通压力都非常大。深圳的早高峰峰值为 13.07%，相对较低，但仍处于较高水平，这可能与其快速发展的经济和不断增加的人口有关。广州的早高峰峰值最低，仅为 7.52%，这说明广州在早高峰时段的交通状况相对较好，可能与当地较为分散的工作区域和灵活的上班时间有关。此外，四个城市的早高峰峰值均出现在 8:30 附近，但北京的峰值略早于上海，这可能与北京的交通流量更早开始集中有关。

- 晚高峰的拥堵情况：

晚高峰时段，上海的拥堵峰值最高，达到 27.02%，这表明上海在晚上交通压力非常大。北京的晚高峰峰值为 25.16%，与广州的 24.77%相近，说明这两个城市的晚高峰拥堵程度也较为严重。深圳的晚高峰峰值最低，为 22.16%，这可能与深圳的产业结构和工作时间安排有关，部分人群可能在晚上有较多的弹性时间，从而缓解了交通压力。值得注意的是，上海的晚高峰持续时间相对更久，这可能是由于上海的商业活动和社交活动较为丰富，人们下班后仍有许多出行需求，导致交通拥堵时间延长。

- 全天路况的综合对比

综合来看，北京的全天路况可能是四个城市中最差的。这不仅是因为其庞大的城市规模和复杂的城市功能，还因为其交通流量在多个时段都处于较高水平。深圳和广州的路况相对较好，这可能与这两个城市近年来在交通基础设施建设和交通管理方面的努力有关。然而，上海晚上的路况可能是四个城市中最差的，这或许从侧面反映了上海的加班文化较为普遍，许多人在晚上仍需出行，从而加剧了交通拥堵。